OrchLang: Replayable Contracts for Token Usage and Request-Equivalent LLM Workflows

Nambi Rajan M · Compiler Design Laboratory

Technical report / research-software paper draft · 9 October 2026

Software repository

Abstract

Equal token budgets do not establish that an LLM workflow's usage is independent of secrets. OrchLang is offline research software separating bounded accounting, request equivalence, and model-specific usage distributions. Its C++17 compiler resolves typed requests and dependent results; a graph checker supports productive cycles and independently replayed product certificates. Rational-kernel tools quantify finite leakage, check sufficient stochastic bisimulation, and decide all-finite-horizon ordered trace-law equality by exact linear algebra. Equality certificates and finite inequality witnesses have independent replay. Narrow source bridges connect admitted OrchLang and actual Python LangGraph programs to replay-gated execution. The compiler admits 15 of 15 balanced source projections, compared with eight previously. Independent bounded semantics find no accepted request or usage difference in 13,824 secret comparisons. Finite-channel analysis agrees with independent enumeration in 260 histogram comparisons, and all 130 saved reports replay at explicit caps. The exact trace-law corpus gives six equality proofs and six finite witnesses, all replayed, including an equal-law case missed by strong bisimulation. A pinned LangGraph backend agrees in 888 offline runs; original restricted Python sources agree in 30 accepted trace comparisons. Examples distinguish equal means from equal laws and aggregate totals from ordered observations. The contribution specializes established methods in an inspectable software pipeline. Results require declared accounting and successful-outcome assumptions. Provider invoices, unrestricted framework translation, termination and whole-cycle budgets remain outside scope.

Keywords: static analysis; relational verification; resource noninterference; LLM orchestration; proof certificates; research software.

1. Research problem and software purpose

An operator may observe a workflow's request usage while lacking access to a confidential routing decision. A conventional budget check answers whether consumption stays below a ceiling. It does not answer whether changing a secret changes the usage record. This distinction matters in multi-call workflows because a secret can affect which public input is requested, which model is used, whether a second call is made, or whether a response-dependent continuation repeats.

Consider public inputs x and y, each bounded by 100 tokens. A secret Boolean selects which is sent to a model. Both arms make one call to the same model using the same template and have identical worst-case bounds. With concrete lengths of 37 and 61 tokens, their input usage differs.

```
workflow EqualBounds budget 1000 {
  input x: text max_tokens 100;
  input y: text max_tokens 100;
  secret route: boolean;
  model m = mock("worker") max_tokens 60;
  prompt step(t: text) -> text = "Process {t}";
  if route { let a: text = call step(x) using m; }
  else { let b: text = call step(y) using m; }
  output "done";
}
```

OrchLang rejects this workflow with `E236`. No secret text appears directly in the request. The distinction arises from secret-controlled selection of different public values. An observer needs the defined usage access and suitable knowledge of those public values; this example does not establish that a conventional invoice exposes individual calls.

Resource-aware noninterference, relational cost analysis, self-composition, and product verification are established [1-3,16-17]. Expected resource bounds for probabilistic programs are established [4]; PPDL models uncertainty in multi-call LLM flows [5]. A recent author-hosted note also treats cumulative LLM-workflow cost bounds [18]. Stochasticity does not prevent numerical resource analysis. Our narrower question is how a usable workflow tool can retain sufficient request identity and dependent-value correspondence to establish equality of a selected usage observation, and how its conclusion can be independently replayed.

The implemented software contributions are:

1. A bounded compiler combining structural token bounds, declared information-flow policy, and an invocation-level relational check over resolved model contracts, ordered request atoms, public guards, bounded retries, and dependent call results.
2. A finite-control graph front end that extends the modeled workflow shape to call-bearing cycles and tracks equal result registers through overwrite and repeated calls. Its guarantee concerns trace prefixes and stop positions, not a finite total budget.
3. Two independent validation components: a source-cost auditor for the compiler's reports and a certificate replayer that reconstructs the graph product closure without importing the producer.
4. A reproducible evaluation package with source provenance, explicit reductions and exclusions, independent full-text bounded semantics, negative cases, conservative rejections, and separate graph value probes.
5. Exact finite rational usage channels with complete values and bounded outcome history, independently reconstructed histograms and exact quantitative information-flow facts.
6. A finite stochastic transducer with exact partition refinement and independent closure replay, providing a sufficient equality result for every finite call horizon under a supplied model.
7. Exact rational row-span checking that decides ordered usage-law equivalence for the supplied finite model and selected secret stores, with independently replayable equality certificates and finite inequality witnesses.
8. Narrow OrchLang and actual Python LangGraph source adapters, a replay-gated LangGraph runtime, an isolated-tested analysis wheel and a functioning HTML demonstration.

The new software arrangement specializes established relational reasoning to model-request contracts. It does not introduce a new general theory of noninterference, bisimulation, or probabilistic coupling. Normalizing model aliases and repairing literal identity are correctness changes, rather than independent novelty claims.

2. Software architecture and supported fragments

2.1 Bounded OrchLang compiler

The C++17 implementation includes a lexer, recursive-descent parser, AST, scoped symbol table, semantic analysis, cost analysis, relational analysis, dependency IR, JSON reports, and an offline interpreter. Workflows declare budgets, bounded inputs and secrets, models, typed prompts, and tools. Statements include model calls, branches, literal-bounded `retry` blocks, output, effects, and explicit declassification or endorsement with a written reason. Blocks have lexical scope. The language has no general mutable assignment, recursion, dynamic dispatch, concurrency, or unbounded loop.

Normal `check`, report generation, and execution require the semantic, cost, and relational gates. `--audit-unsafe` is an explicit diagnostic bypass for otherwise well-typed cost/relational rejections; input contracts remain checked. The mock interpreter carries token counts and Boolean flags, rather than generated provider text. It does not call providers or execute external tools.

The public CLI supports checking, certification, dependency IR, and seeded execution. `docs/LANGUAGE_SPEC.md` gives the grammar, diagnostics, accounting premises, and release policies. `docs/REVIEW_DEMO.md` provides reproducible counterexamples and accepted dependent chains.

2.2 Experimental finite-control graph front end

The separate Python front end consumes strict versioned JSON, rather than adding cycle syntax to OrchLang. A graph declares bounded inputs, resolved model handles, text registers, states, and an entry state. Each state contains an ordered list of calls followed by a `goto`, public choice, secret choice, or stop. A call specifies a model, exact template, ordered literal/value arguments, and a destination register. Public guards test Boolean inputs or token counts of text values using a specified comparison and threshold.

Registers are mutable and associated with a producer token unit and cap. Overwrite is modeled explicitly. Private inputs may select control alternatives but cannot appear directly in request arguments. Register arguments must remain in compatible provider/model token units. The front end rejects unknown fields, duplicate JSON keys, nonfinite prices, malformed references, and reachable call-free cycles. A resource cap produces `unknown`, never acceptance.

Call-bearing cycles may continue indefinitely. State-entry cost summaries bound a single execution of a state's calls under the declared contracts. Summing them across an unbounded cycle does not yield a finite workflow budget. The graph front end supports no tools, deliberate releases, request serialization, or verified framework extraction. Its schema, tutorial, and exact limits are in `docs/CYCLIC_WORKFLOWS.md`.

2.3 Artifacts and independent checks

Compiler certificate v3 binds raw source bytes by SHA-256 and records resolved call facts, cost derivations, labels, trusted releases, and local relational obligations. The source-cost auditor has its own lexer, parser, lexical resolver, and cost semantics. It compares source-derived model/request facts, all cost components, and every recorded derivation. It does not verify typing, information-flow labels, relational obligations, or the truth of a release justification.

An accepted graph certificate records product nodes `(pcL, pcR, R)`, local rules, required successors, reachable source states, and per-state cost summaries. The separate replayer strictly parses both inputs, binds the graph bytes, reconstructs the product from its public-input root, and checks every node, relation, rule, universal edge, count, and declared resource cap. It imports no producer code. The replayer attests its own hash and certificate/source hashes; a copied producer

hash is explicitly unverified. This is independent closure checking, not a mechanized proof of the replayer's metatheory.

Hashes bind exact bytes and make stale evidence detectable. They do not establish authorship, authenticate a publisher, or prove either implementation correct.

2.4 Source bridges and actual LangGraph execution

A bounded source bridge exports admitted acyclic OrchLang programs after gating an immutable snapshot of the parsed source with a snapshot of the compiler executable. It retains lexical binding, exact templates, dependent calls and guards, with a final unconditional literal or public-text identifier output. Retries, tools, explicit releases and unsupported syntax fail closed. Source mappings and hashes bind the source, compiler, adapter, graph and independently replayed certificate. It is a translation implementation, not a verified compiler.

A second frontend reads actual Python `StateGraph` source with Python's AST parser. Its restricted subset includes a static graph builder, synchronous straight-line request functions, typed bounded state fields, deterministic Boolean/token routing, static edges and productive cycles. Contracts supply input labels/bounds and model declarations; calls and edges come from the source rather than a manually authored graph. The extractor rejects dynamic control, state reducers, fan-out, effects, copied outputs and unsupported expressions. Because LangGraph applies returned state updates atomically, an early extracted register update is allowed only when subsequent calls do not read the overwritten state's old value. Accepted exports are independently replayed before execution.

A replay-gated backend executes the graph using pinned LangGraph 1.2.14 in an isolated environment. It validates immutable graph/certificate bytes, concrete input bounds and call caps. The source study also executes the original Python `StateGraph` functions with the same offline request session. These are authored restricted programs; they do not establish general Python translation or production provider behavior.

3. Contracts, observer, and conditional guarantees

3.1 Resolved models and complete modeled requests

A resolved model contract is `M = (provider, model ID, output cap, price-present, unit price)`. Aliases do not determine identity. Two aliases may resolve to one contract; a shadowed alias may resolve to another. Price is finite normalized configuration metadata. Token claims do not establish exact floating-point monetary bills or currency invoice equality.

The abstract request is $Q = (T, a_1, \ldots, a_n)$: exact decoded template bytes and ordered argument values. In the compiler, static atoms denote typed literal kind plus decoded bytes, lexically resolved outer values, or earlier aligned call results. Boolean `true` and text `"true"` remain distinct abstract request atoms, matching the request-sensitive mock. In the graph checker, atoms denote literals or related input/register values. Equal token estimates are never treated as equal requests.

These fields cover request-affecting information expressible in the supported models. Provider system messages, tool schemas, decoding parameters, cached context, deployment changes, hidden routing, and serialization overhead are absent. A claim about a provider must include every relevant request field and state in the contract.

A generated value's bound is associated with its producing provider/model tokenizer identity. Cross-identity use is rejected by compiler diagnostic `E267` and by the graph schema. There is no conversion contract in this release. Different caps on the same canonical model use compatible modeled units; identity alone does not prove a physical tokenizer bound.

3.2 Unary accounting premises

Each admitted bounded execution must satisfy:

- Output usage for every call is at most its declared cap.
- Complete modeled input usage is at most $\tau_d(T) + \Sigma\, b(a_i)$, with $\tau_d(T)=\mathrm{ceil}(\mathrm{UTF8_bytes}(T)/d)$ and argument bounds in compatible target units.
- External inputs respect declarations and retry attempts do not exceed their literal maximum.

The CLI name `chars-per-token` is historical; the estimator uses UTF-8 bytes. Default `d=4` is an explicit approximation contract. Even `d=1` is not an unconditional provider bound because request-envelope overhead, tokenizer boundaries, and generated-value conversion remain outside the model. The mock implements additive fragment accounting directly. Runtime input pins enforce names, types, and bounds.

The cost triple `C=(G,I,U)` bounds output, input, and total usage independently. Historical report names are `guaranteed_tokens`, `estimated_tokens`, and `total_tokens`. A call contributes `(cap,template-estimate+Σargument-bound,cap+input-bound)`. Sequence adds componentwise; a branch takes componentwise maxima; `retry n` multiplies by `n`. Different arms can dominate different components, so $U \le G+I$ may be strict. Saturation is a top sentinel indicating an unusable bound, and `E266` rejects it; it is not a representable mathematical upper bound on an arbitrary-size integer cost.

Conditional unary bound. An admitted bounded run satisfying these premises uses at most `G` output, `I` input, and `U` total tokens. Structural induction follows sequence addition, componentwise branch dominance, and bounded repetition. This is a proof sketch, not a machine-checked theorem. Graph state summaries use the same accounting premises only for a single state entry.

3.3 Usage observation and shared complete-outcome oracle

The billing observation `B` groups usage by resolved model contract into `(call count,input tokens,output tokens)`. Equality of these vectors implies equality after aggregation by canonical model. Both analyzers establish a stronger ordered invocation alignment. For an infinite graph execution, observations are defined on finite call prefixes; an infinite aggregate total is not a finite bill.

Let `Ω(M,k,Q)` return a complete outcome, including response value, usage, and future-relevant branch or retry information. Paired executions share this oracle and corresponding retry-success events. Equal `(M,k,Q)` yields equal outcomes. The per-contract ordinal `k` agrees when earlier aligned events agree. This is a pathwise coupling premise; independently sampled provider executions need not return identical answers.

The C++ ordinal mock keys outcomes by workflow, seed, resolved contract, and ordinal. The separate request-sensitive mode keys by seed, resolved contract, ordinal, exact template/typed-literal fields, and identifier projections `(tokens,flag)`; that mode omits the workflow name. It still carries no complete response text. The separate bounded Python reference and graph probe use full concrete strings and request-sensitive complete outcomes. All remain offline diagnostic models.

Conditional stochastic-kernel corollary. Suppose corresponding calls have the same conditional kernel over complete outcomes given their resolved contracts, ordinals, full requests, and aligned relevant history. An identity coupling can choose the same complete outcome at each matched call. The pathwise argument then gives equality of usage-prefix laws; bounded terminating workflows also have equal complete usage laws. Independence across calls is unnecessary if the conditioning history captures their dependence. Hidden account, session,

provider, or cache state must be aligned or included, otherwise the premise fails. This is standard coupling reasoning [19-21], not an implemented provider model or a new theorem of probability. It does not assert pathwise equality between independently sampled runs.

3.4 Declared flow policy and release boundary

The compiler lattice is `{Public≤Secret}×{Trusted≤Untrusted}`. It distinguishes explicit-data label `D` from effective label `L`, which includes the program counter. A call computes `D(result)=join D(arguments)` and `L(result)=pc join join L(arguments)`. Direct secret request data is rejected with `E230`; outputs and tools retain the effective policy. Tools also require trusted values and control.

Explicit-data labels alone cannot establish secret independence: a secret may select different public requests. The relational gate is essential. Conversely, using only the effective label prevents safe dependent calls inside request-equivalent private alternatives. The combination admits those calls while preserving strict output/effect checks.

The whole bounded billing theorem requires no declassification. A deliberate release can change usage even when a local alternative is structurally discharged. Certificate v3 records releases and withholds the unrestricted claim in their presence. Endorsement changes integrity, not confidentiality. Written reasons are trusted assertions, not executed validators. The graph front end excludes releases entirely. Neither tool proves prompt-injection elimination or malicious-text detection.

4. Relational analysis and replayable graph closure

4.1 Bounded invocation signatures

The compiler compares ordered signatures of calls, bounded retries, and public choices. Call atoms retain exact templates, literals, resolved outer identities, and earlier matched-result positions. Public choices retain resolved guard subjects, comparison operators, thresholds, and arm signatures. Secret-data choices require matching defined alternatives. Lexical environments are copied into branches/retries; corresponding regions reserve aligned result positions.

```
if route {
  let a: text = call step(query) using m;
  let b: text = call step(a) using m;
} else {
  let x: text = call step(query) using m;
  let y: text = call step(x) using m;
}
```

The preceding compiler rejected the second calls because `a` and `x` inherited secret control. The combined analysis admits them: the first requests match, their coupled outcomes match, and the second calls use corresponding earlier results. Effective labels still prevent secret-controlled output or tool effects.

Conditional bounded billing equality. Two accepted bounded runs with equal public initial values, no declassification, valid accounting premises, and shared complete-outcome/retry coupling have equal ordered modeled requests and billing vectors. Induction maintains equal outer and matched-result values. Public guards then agree; secret alternatives may differ but retain matched signatures. Coupled retries preserve attempt counts and ordinals. `docs/FORMAL_CONTRACT.md` states the detailed premises.

4.2 Product states for call-bearing cycles

The graph checker starts from `(entry,entry,R₀)`, with `R₀` relating each public input to itself. A program counter is a state and a call index. `R` is a sufficient equal-value relation between variables in the left and right executions. The rules are:

- A unilateral `goto` advances one counter without changing `R`.
- A secret decision produces obligations for both alternatives. Expanding each side covers all paired private choices, including combinations impossible for one fixed secret valuation; this is conservative.
- Public decisions require related subjects and identical predicates, then pair corresponding outcomes.
- Calls require equal resolved contracts, exact templates, and pairwise equal literals or `R`-related arguments.
- When matched calls write `l` and `r`, remove every relation with left endpoint `l` or right endpoint `r`, then add `(l,r)`. This prevents stale equalities after overwrite.
- Stops must match. A call versus stop, incompatible public guard, or unequal request obligation rejects the candidate.

A breadth-first exploration reuses previously visited product states. The certificate is the finite closed relation reached from the prescribed root, rather than a collection of user-supplied local assertions. Every required successor must be present. The independent replayer recomputes this closure from the graph and compares the certificate's complete contents.

A representative graph selects one of two two-call lanes using a secret. Each lane calls `Process {value}` on the same public query, then calls it on its own first result. The second result controls whether the graph repeats. Although register names differ, their values are related after each matched call. The accepted cyclic fixture has 91 product nodes and 114 edges. A split-block variant, with an intervening `goto`, also verifies. A stale-result variant overwrites one side's first result but reuses the other side's old first result in a later request; relation killing correctly rejects it.

Conditional graph property. For accepted graphs with equal public initial values and the shared complete-outcome/accounting contracts, every finite aligned call prefix has equal requests, outcomes, and usage, and stop positions agree. The invariant interprets every pair in `R` as equal concrete values. Matched calls preserve it by oracle equality and assignment killing; public rules preserve common guards; private rules cover all choices. Certificate closure supplies the coinductive step for revisited states. Excluding reachable call-free cycles prevents an infinite silent run from hiding a termination distinction. Call-bearing cycles can remain infinite, so termination and finite whole-execution cost are not conclusions.

This finite product and coinductive argument follow established self-composition and relational verification [16-17]. The applied software contribution is its request-contract specialization, inspectable overwrite relation, and independent replay pipeline. The sketch is not mechanized and the graph schema is not a proof of arbitrary framework semantics.

4.3 Precision and resource limits

Both checks are sufficient and incomplete. The compiler rejects equivalent guards written `<11` versus `<=10`, templates differing only in placeholder names, and distinct outer bindings whose concrete values happen to agree. Exact literal distinctions protect request identity even when byte estimates agree. The graph checker additionally explores all private branch combinations and excludes call-free cycles, including some harmless ones. Exhausting the product cap returns `unknown`.

Diagnostic value probes enforce typed input stores, valid scalar UTF-8, integer seeds and call limits, and explicit byte/control budgets. These probe limits prevent unbounded allocations; they do not change the accepted product theorem. Stop/control events are inspected after the last permitted call, so a run ending exactly at the horizon is recorded as stopped rather than truncated.

4.4 Exact finite success-only usage channels

The structural method is distribution-free under its shared-outcome premise but can conservatively reject different requests whose usage laws coincide under one model. A separate quantitative analyzer consumes an explicit finite rational kernel `K(C,k,Q,h_w)` over `(response text, output usage)`, where `h_w` is a 0-4 event complete history window. Rules use exact optional-field conjunction and first-match precedence; otherwise the contract default applies. The declared model excludes provider state outside this history, failures, native retries, cache discounts and currency.

For a fixed public store and 2-64 distinct secret assignments with explicit priors, exact dynamic programming computes either a per-contract aggregate-usage vector or an ordered usage-only trace, each with a stop bit. The state retains the control position, all initialized values, per-contract ordinals, accumulated observation, finite history and trace bytes. Equal complete states merge exact probability mass. After the call horizon, control continues to a stop or the next call; the latter is a truncated prefix. Resource exhaustion yields unknown without a partial histogram or leakage conclusion.

An induction on call depth establishes that each DP mass is the sum of probabilities of concrete prefixes having that complete state. Deterministic normalization preserves mass; unit-mass conditional kernels distribute it; identical states have identical future control and kernels; observation projection sums terminal masses. Completion requires total mass exactly one. The implementation has explicit state, value, trace, cumulative serialization-work and rational-precision caps.

The outputs include exact pairwise total variation, symmetric likelihood ratios with an infinite flag, posterior Bayes vulnerability and its prior-relative ratio. Mutual information is retained as a finite expression with rational coefficients and logarithm arguments; displayed logarithms are floating approximations. No differential privacy guarantee is claimed without an adjacency relation. An independent parser and no-merge DFS evaluator re-executes small cases. These quantitative information-flow metrics and algorithms are established [22-24]. The application delta is a declared complete-request/history-indexed usage observer connected to source and replay evidence. A zero-law result is conditional on the selected store, secrets and kernel, not a replacement for universal request equivalence.

4.5 Finite stochastic closure for every finite horizon

The same finite rational kernel also supports a sufficient result without fixing a call horizon. The semantic state is `(pc, values, ordinals, history)`, normalized to the next call or stop. It retains all initialized values, including hidden registers that may affect later requests or guards. It omits accumulated usage and cumulative execution-work monitors: these monitors do not belong to the ideal successful-outcome semantics proved here.

Kernel version 1 tests literal call ordinals and history ordinals only from 0 through 32. Replacing every ordinal at or above 33 by 33 preserves which rules match, including retained-history tests. The fixed inputs, finite outcome alphabet, finite control, saturated ordinals and history window of at most four events therefore yield a finite reachable state system. The quotient can still be large; it is not a polynomial-size or practical-termination guarantee for the analyzer.

A visible transition label is `(resolved contract, input usage, output usage)`. The checker builds the union of the states reachable from the explicit secret assignments, using exact rational edge masses. It refines an initial terminal/call partition by distributions over `(visible label, target block)` until no block splits. For a stable partition, states `s` and `t` in one block satisfy, for every visible label `l` and block `B`:

```
sum(P(s,l,u) for u in B) = sum(P(t,l,u) for u in B).
```

Conditional finite-horizon theorem. Initial states in one stable block induce equal ordered usage-trace distributions at every finite call horizon, including the stop/truncation bit. For horizon zero, normalized terminal classes agree. Inductively, stability gives the same mass for each next usage label and successor block; the induction hypothesis gives the same remaining trace distribution within that block. Summing these distributions establishes the next horizon. This is a specialization of established strong probabilistic bisimulation [26], not a new general probability theorem.

Different initial blocks yield **inconclusive**: strong bisimulation is sufficient, not complete, for trace-law equality. No leakage witness follows from a split. State, transition, control, byte-work or rational-precision exhaustion yields **unknown** without a partial closure or partition claim. These construction limits do not enter the proved semantic state. The theorem excludes cumulative runtime resource monitors, errors/retries, currency, provider probability validity, infinite-run laws, termination and finite whole-cycle cost.

The independent stochastic replayer imports the independent parser/reference, rather than the producer, graph checker or channel semantics. It reconstructs the finite closure, verifies exact transition masses and source/input bindings, and checks a sound stable partition (which need not be the coarsest). An inconclusive certificate can be verified structurally without an equality claim. Producer statistics and a copied checker digest are advisory. Full details are in `docs/STOCHASTIC_BISIMULATION.md`.

4.6 Exact ordered trace-law equivalence

Strong probabilistic bisimulation can separate hidden states even when their observable usage laws agree. The exact trace-equivalence tool applies established probabilistic/weighted-automaton equivalence [27-29] to the same finite semantic system. APEX previously translated probabilistic programs to automata and checked equivalence for properties including anonymity [30]; the present tool specializes that approach to declared LLM usage observations.

For a fixed public store and each selected secret scenario `i` against scenario 0, let `μ_i = e_i - e_0`. For each visible label `l`, the rational matrix `A_l[s,t]` contains its transition probability. The columns `b_S` and `b_N` indicate stopped and nonterminal states. For a finite usage word `w`, the two exact

differences are `μ_i A_w b_S` and `μ_i A_w b_N`. Keeping both distinguishes successful stopping from a still-live call-cap prefix.

The checker explores the reachable rational row span generated by `μ_i A_w`, retaining independent rows and their concrete words. If every basis row annihilates both columns, and the span is closed under every label matrix, all finite words do so by induction. Conversely, a nonzero output is a finite observation with differing probabilities, independently recomputed from each initial state. The finite dimension bounds the number of independent rows; absent resource exhaustion, this decides equality for the supplied finite model, not only a sufficient structural relation.

An equality certificate records the state order, reachable-word rows, exact rational values, rank, initial membership, matrix closure and zero outputs for every required scenario pair. The independent replayer reconstructs semantics through the separately implemented stochastic reference and implements its own rational elimination and closure checks. An inequality report needs only one exact distinguishing pair and word. It does not convert a bisimulation split into a witness. Source/kernel/input hashes bind both claims; producer statistics and copied executable digests are not semantic verification.

State, matrix-entry, byte-work and rational-precision caps yield **unknown** without partial closure, span or decision. The work meters are not RSS or wall-clock guarantees. The comparison remains successful calls for listed private stores, fixed public inputs and the supplied finite kernel. Errors, retries, cumulative monitors, provider probability validity, infinite-run laws and termination probabilities remain excluded. See `docs/TRACE_EQUIVALENCE.md` for the interface, proof argument and operational budgets.

5. Evaluation protocol and results

5.1 Evidence families and provenance

The evidence families address different claims. Development grids check repairs and authored policies; the source study examines partial projections from three richer workflow languages. Independent bounded full-text semantics corroborate accepted C++ verdicts. Graph studies check certificate closure and finite value behavior. Source/runtime studies test restricted translation and actual LangGraph execution. Finite quantitative and stochastic studies compare exact supplied-model laws and replayed closures. Repeated seeds, inputs and horizons are not independent applications.

The source study contains 15 pinned files from PDL, APPL, and LMQL [7-9]. Each yields a base reduction, balanced private-routing variant, and extra-call mutation, for 45 authored source files. `bench/sourced/manifest.json` records exact revisions, URLs, hashes, retained structure, removed semantics, added contracts, and projection digests. A separate catalog has 17 candidate files from four repositories; two were not used as implemented projections.

These are partial bounded abstractions, not faithful complete application ports. Dynamic tool execution, real retrieval, parallelism, constrained decoding, unbounded loops, and growing context are excluded. A ReAct example becomes a capped request skeleton. Alternating actors use one canonical mock model because cross-tokenizer conversion is unsupported. Selection, labels, reductions, and mutations were author-performed after language design. There is no held-out annotation claim, prevalence estimate, external user study, or allegation of upstream vulnerabilities.

5.2 Bounded compiler and source study

Experiment	Observed result
Compiler regressions and sanitizer	112/112 tests pass; ASan/UBSan and shipped examples pass
Independent source-cost auditor	21/21 tests pass
Development cost grid	0 component/total violations in 4,600 runs
Flat call-sum / branch-aware retry-once baselines	741 / 754 violations (16.1% / 16.4%)
Certified total / peak sampled total	Median 1.20×; range 1.03-1.75×
Accepted development relational cases	0 billing differences in 2,975 comparisons
Leaking development cases	All seven have concrete witnesses
Policy and conservative cases	13 expected policy pairs; 300 conservative pairs with equal ordinal-mock bills
Base source projections, each mock mode	0 bound violations in 1,500 runs
Balanced source projections, each mock mode	0 billing differences in 2,250 comparisons
Extra-call source mutations, each mock mode	All 15 rejected; all 2,250 slices differ
Balanced files on preceding / upgraded compiler	8/15 / 15/15 accepted
Source-bound report audit, each mode	30/30 pass

The preceding version is pinned at `a49fcdbdaa99723d7b6e1913f6d01e4f88682dd9`; acceptance comparisons use identical files. The runtime now uses contract/ordinal keys rather than a sequential random stream, so sampled peak/violation statistics differ from historical reports. The old `SameLiteral` case used different strings of equal estimate; it remains a conservative rejection, while the accepted case uses identical strings.

Ordinal and request-sensitive source studies use the same fixed grid and are stored separately. They are not pooled as independent observations. Actual source-located branch decisions are recorded and the development grid covers all tracked arms. Timing includes process startup and local load; it is not an isolated analyzer benchmark. The engineering baselines are transparent heuristics, rather than implementations of AARA, RelCost, PPDL, or other research systems.

5.3 Independent full-text bounded stress

A separate Python implementation generates 264 bounded cases: 144 positives over six structural shapes and 120 mutations over five classes. It renders source from a generated AST and interprets that AST independently of C++ runtime traces and compiler certificates. Its request-sensitive oracle carries complete response strings, exact ordered argument values, and additive UTF-8 accounting.

All 144 positives pass C++ `check`; all 120 mutations are rejected with `E236`. Across 24,192 accepted executions, 13,824 secret comparisons, 66,240 calls/accounting checks, and 5,064 retry attempts, no accepted request or usage difference occurs. Negative probes find finite request/usage witnesses for 117 cases. Three retry-bound mutations have no witness on the finite grid and remain unknown; they are not classified as false positives. An equal-value argument subgrid yields

1,728 comparisons with no witness even though different lexical bindings are conservatively rejected.

The reference shares the generator AST with source rendering. It is independent semantic validation against the compiler, but not an independent source parser, exhaustive language analysis, mechanized proof, provider study, or new external benchmark. Exact case and oracle manifests are in `bench/results/relational_stress.json` and `docs/RELATIONAL_STRESS.md`.

5.4 Cyclic fixtures and independent replay

The authored graph corpus has six accepted, eight rejected, and three invalid fixtures. Accepted shapes cover dependent cyclic chains, canonical aliases, split control blocks, nested private decisions, an intentionally nonterminating productive cycle, and a finite private chain. Negative cases mutate templates, model caps, outer arguments, termination/call count, public guards, overwrite relations, or literals. Invalid cases include silent private cycles, direct secret requests, and incompatible generated token units.

All fixture verdicts match expectations. Every accepted certificate independently replays; closure sizes range from 15 to 206 nodes and 14 to 274 edges. The value grid uses 25 seeds, four public text values including UTF-8, all Boolean secret assignments, and at most 120 calls per execution. It records 1,400 accepted runs, 800 secret comparisons, and 28,580 accounting checks, with zero full-observation, request, or billing-vector differences. Each of the eight rejected fixtures has both a full-observation and billing-vector witness somewhere on this grid. A witness is a complete-run observation only when its saved stop flag is true; otherwise it concerns the finite call prefix. No finite total bill is asserted for an infinite cycle.

The product and replayer implementations are separate. The graph value probe shares the producer's schema code, so it is not an independent interpreter. Thirty-three replayer tests cover accepted closure and malformed/tampered schemas, roots, relations, calls, edges, counts, caps, stop rules, and source hashes. The producer has 30 focused tests, including resource/UTF-8 robustness; two observation tests distinguish request differences from billing differences.

A seeded 3,000-small-graph property probe records 859 constructor rejections, 1,462 checker acceptances, and 679 checker rejections. Its 5,848 distinct-secret comparisons find no accepted request, usage, response, or stop differences at a 20-call horizon. The 11,696 ordered comparisons also include equal-secret pairs. This probe uses the producer's own value semantics and demonstrates internal consistency, not independent soundness. Complete seeds, graph hashes, truncation counts, and environment data accompany the report.

5.5 Source translation and LangGraph conformance

Of 20 existing accepted OrchLang programs (five research examples and 15 balanced source projections), 17 export; three retry examples are explicitly excluded. Across 720 source-AST/graph run pairs, 360 secret-trace pairs and 1,584 call events, no request, usage or stop discrepancy occurs. The independently interpreted source AST shares the source parser with lowering, so this is finite translation evidence rather than an independent-parser proof.

The actual LangGraph backend matches the graph reference in 888 offline executions over six authored graphs and the 17 exports, with 4,528 call-event pairs and no trace discrepancy. Of these executions, 864 stop and 24 reach the call cap; they are not claimed to terminate. Eleven backend tests cover replay gating, mutable-handle tampering, limits and input validation.

The Python source frontend additionally records 36 actual original-source executions: 30 accepted source/backend comparisons all match, and six rejected-graph source/value probes all match. It records 66 calls, 12 equal accepted secret-assignment pairs and three intentionally different pairs for a rejected secret-template case. Direct secret arguments and an atomic-update mismatch are

rejected during extraction. Twenty frontend tests include stale-attestation invalidation when output paths are reused. This is evidence for authored fixtures and their stated subset, not external adoption or unrestricted framework extraction.

5.6 Controlled probabilistic usage study

Eleven authored graph/kernel pairs exercise conditional kernels, dependent results, alias normalization, productive cycles, stop/truncation, public-store dependence and observer choice. Three graphs pass structural request checking; eight fail it. Selected horizons and two observers produce 130 two-secret analyses and 260 exact DP/independent-DFS histogram comparisons. All compared distributions and unit masses agree. All 11 state-cap probes return unknown without a histogram or metric.

A hand-calculated noisy binary usage channel has total variation 1/2 and posterior vulnerability 3/4 under a uniform prior. Another case produces output usage 1 or 3 with equal probability in one branch and deterministic usage 2 in the other: expected totals agree but total variation is one. A two-call case has deterministic usage sequences `(1,3)` and `(3,1)`: aggregate TV is zero, ordered-usage TV is one. History-conditioned rules can expose a later difference despite equal first-call usage. Other fixtures show that fixed-store equality does not generalize to different public inputs. Seventeen analyses have equal expected totals but positive TV; none is dismissed as safe by an expected-cost comparison.

Across this constructed corpus the DP expands 1,132 outcome transitions and the no-merge reference 35,692, with 18,056 leaves. These work counts illustrate merging on small controlled cases and are not a real-world performance benchmark. Kernels are supplied synthetic distributions, not learned provider probabilities. The independent report replayer checks input bindings, exact histograms and exact leakage expressions; producer counters and approximate logarithms are excluded from its attestation.

All 130 saved reports independently replay at explicit caps of one million reference paths, ten million work steps and one GiB deterministic byte work. Default caps of 100,000 paths, one million steps and 64 MiB give 129 verified reports and one unknown (the largest trace case). The byte budget measures canonical serialization work, not process RSS. `bench/results/probabilistic_replay.json` records every report/attestation digest and both cap configurations.

5.7 Stochastic closure and independent replay

The same 11 graph/kernel pairs yield five **proved_equal**, six **inconclusive**, and no normal-run unknown results. All 11 certificates independently replay against their exact graph, kernel and input bytes. Every fixture also produces a fresh unknown certificate at `max_states=1` and a fresh independent unknown attestation; no prior successful evidence is reused.

The five equality cases pass 60 exact DP checks across horizons 1, 2, 4, 8, 16 and 32 and both aggregate and ordered observers, covering 120 unit-mass scenario histograms. These finite comparisons use the channel DP and corroborate implementation behavior. The stable partition, checked independently, supports the conclusion for every finite horizon. It is not inferred from those six sampled horizons. The driver deterministically reproduces the summary and all 60 saved evidence files. `bench/results/stochastic_bisimulation.json` binds the source, kernel, expected stores/secrets, certificates, attestations and DP results.

5.8 Exact trace-law study

The exact checker analyzes the eleven existing graph/kernel pairs and one new authored equal-but-not-bisimilar case. It yields six **proved_equal** decisions and six **unequal** decisions with finite word witnesses. All twelve independently replay. The equality cases pass 48 finite DP comparisons (96 unit-mass scenario histograms) at horizons 1, 2, 4 and 8 under aggregate and ordered observers. All six witness probabilities agree with DP at the corresponding finite horizon. These finite checks corroborate the implementations; span closure supports the every-finite-horizon equality result.

Every case also generates a fresh state-cap **unknown** report and independent unknown attestation. Twelve semantically tampered reports are independently rejected. `bench/results/trace_equivalence.json` records decisions, input/report hashes, replay, comparison with strong bisimulation, DP corroboration, tampering and cap behavior.

The new example gives a hidden first response red or blue with probability one half and identical first usage. One private arm chooses deterministic second usage from this response; the other uses an independent mixture with the same marginal law. Both ordered usage laws are equal. Strong bisimulation is inconclusive, while exact row-span checking proves equality. This demonstrates a concrete precision gain in an authored finite model. It supplies neither external adoption nor a new automata-equivalence algorithm.

5.9 Reproduction and evidence limits

`audit/reproduce_final.py --sanitize --package` builds the compiler, runs regressions and demo HTTP tests, checks examples, locally builds the pinned preceding compiler, and repeats bounded, graph, source and quantitative evidence plus isolated wheel installation. `--integration-python integrations/langgraph/.venv/bin/python` also runs the pinned actual backend/source tests and studies; the script never installs that environment. The final manifest records commands, return codes, environment, input/source hashes and result digests. It rejects executable inputs that changed during the run. Long studies are omitted only under explicit `--quick`, which is marked in the manifest.

Zero sampled differences do not prove soundness. All provider behavior is mocked; complete request serialization, invoice access, answer quality, timing leakage, prompt-injection success, deployment prevalence, and independent provider distributions are unmeasured. The graph model has no verified extraction from an arbitrary framework. The source-cost auditor and independent graph replayer check different claims. Neither should be advertised as a universal safety-proof verifier.

For a fixed public-input/shared-oracle slice and uniform Boolean secret, distinct deterministic billing observations reveal one bit, and identical observations reveal zero bits [6]. The source study's balanced/extra-call variants therefore have a toy conditional calculation of 0/1 bit. This is not an estimate of real attack capacity or distributional leakage from provider invoices.

6. Related work and contribution boundaries

Constant-resource types combine resource reasoning and information flow [1]; RelCost and monadic refinements compare executions and their costs [2-3]. Self-composition and product programs provide standard foundations for two-run properties [16-17]. OrchLang's unary rule, two-run product, and cycle closure do not independently establish novelty.

Bounded Expectations analyzes probabilistic resources [4]; PPDL represents uncertain LLM flows and already uses token budgets and usage logging [5]. Dickerson's May 2026 author-hosted note describes Petri-net bounds on cumulative LLM-agent resources [18]; it was unrefereed in this audit

and its empirical statements remain author-reported. Coupling/pRHL foundations explain the conditional kernel corollary [19-21]. Unary ceilings, expected costs, pathwise coupled equality, and equality in distribution are distinct properties.

LMQL, APPL, and PDL offer richer prompt/workflow programming [7-9]. CaMeL and FIDES use flow/control policies for agent security [10-11]; AgentFlow and Agentproof include static policy or graph verification [12-13]. Agent verification is therefore neither absent nor exclusively dynamic. Token-length and timing studies motivate precise observer models [14-15], but their traffic/timing observations differ from the selected usage record.

The surviving contribution is focused: a request-aware relational analysis and reusable certificate pipeline that retains model identity, exact ordered request atoms, dependent result correspondence, overwrite handling, and cycle progress conditions. It provides executable boundaries and independent replay rather than claiming that a token cap, content label, or graph alone proves real-provider privacy. The prior-art matrix and source catalog make this distinction reviewable. A systematic priority search, externally reused workflows, verified provider envelopes, and mechanized soundness remain open.

Quantitative information flow already provides Bayes-vulnerability and entropy-based leakage measures [22-23]; exact symbolic information measures for discrete probabilistic programs are also established [24]. The finite channel analyzer specializes these methods to modeled usage observations rather than claiming new metrics. Current concerns about opaque-service token charging motivate careful observability [25], but OrchLang does not attest an external provider or detect dishonest invoices.

Probabilistic automata equivalence, rational weighted-language methods, and program-to-automata security analysis precede this work [27-30]. The exact span checker closes a documented precision gap of the sufficient bisimulation checker within OrchLang's finite model. Exact fractions, usage-label encoding, independent replay and finite witness presentation are software implementation choices, not a new general decision procedure.

7. Software publication and future work

Science of Computer Programming's Software Track / Original Software Publication is the primary submission direction because the artifact is a programming-language and analysis tool. The publisher describes original software publications as reports on existing useful software; usefulness, availability, reproducibility, and clear supported behavior should lead the manuscript. The current draft is a technical report for human review, not a submitted or accepted paper. A software-paper route does not remove the need for an original useful contribution or accurate evidence.

The repository has an MIT license, third-party notices, citation metadata, command-line tutorials, source provenance, negative examples, independent checks, and a local versioned candidate. The authors must verify the current journal guide, complete identity/affiliation/contact and declarations, review the AI-assisted changes, and approve the final submission. No author approval, editor contact, journal acceptance, or acceptance probability is claimed.

Future work should prioritize verified request envelopes and token-unit conversions, independent application use, normalization of equivalent guards/templates, and mechanization of the product invariant. A provider study must measure available usage observations and test kernel/accounting premises without conflating independent sampling with shared pathwise outcomes. Richer quantitative analysis should model errors, retries and hidden provider state and validate fitted kernels. The current exact tools establish facts only for supplied finite successful-outcome models.

References

1. Ngo, Dehesa-Azuara, Fredrikson & Hoffmann. *Verifying and Synthesizing Constant-Resource Implementations with Types*. IEEE S&P 2017. DOI.
2. Çiçek et al. *Relational Cost Analysis*. POPL 2017. DOI.
3. Radiček et al. *Monadic Refinements for Relational Cost Analysis*. POPL 2018. DOI.
4. Ngo, Carbonneaux & Hoffmann. *Bounded Expectations: Resource Analysis for Probabilistic Programs*. PLDI 2018. DOI.
5. Mandel, Baudart, Vaziri & Hirzel. *PPDL: LLM-Based Flows as Probabilistic Programs*. ICML 2026, PMLR 306:86172-86197. Proceedings.
6. Köpf & Basin. *An Information-Theoretic Model for Adaptive Side-Channel Attacks*. CCS 2007. DOI.
7. Beurer-Kellner, Fischer & Vechev. *Prompting Is Programming*. PLDI 2023. DOI.
8. *APPL: A Prompt Programming Language*. ACL 2025. ACL Anthology.
9. *PDL: A Declarative Prompt Programming Language*. arXiv:2410.19135, preprint as audited.
10. *Defeating Prompt Injections by Design* (CaMeL). arXiv:2503.18813, preprint as audited.
11. *Securing AI Agents with Information-Flow Control* (FIDES). arXiv:2505.23643, preprint as audited.
12. AgentFlow. arXiv:2608.22868, preprint.
13. Agentproof. arXiv:2603.20356, preprint.
14. Weiss et al. *What Was Your Prompt?* USENIX Security 2024. Conference paper.
15. Zhang et al. *Time Will Tell*. arXiv:2412.15431, preprint as audited.
16. Barthe, D'Argenio & Rezk. *Secure Information Flow by Self-Composition*. CSFW 2004. DOI.
17. Barthe et al. *Product Programs and Relational Program Logics*. JLAMP 2016. DOI.
18. Dickerson. *Statically Bounding Resource Accumulation in LLM Agent Workflows: The Cost Case*. Author-hosted note, 18 May 2026; unrefereed as audited. Source.
19. Barthe, Espitau, Grégoire, Hsu, Stefanesco & Strub. *Relational Reasoning via Probabilistic Coupling*. LPAR 2015, pp. 387-401. DOI.
20. Barthe & Hsu. *Probabilistic Couplings from Program Logics*. Chapter 5 in *Foundations of Probabilistic Programming*, Cambridge University Press, 2020, pp. 145-184. DOI.
21. Barthe et al. *Coupling Proofs Are Probabilistic Product Programs*. POPL 2017. DOI.
22. Smith. *On the Foundations of Quantitative Information Flow*. FOSSACS 2009. DOI.
23. Braun, Chatzikokolakis & Palamidessi. *Quantitative Notions of Leakage for One-try Attacks*. ENTCS 2009. DOI.
24. Schröer, Randone, Pardo & Wąsowski. *Symbolic Quantitative Information Flow for Probabilistic Programs*. 2024. DOI.
25. Sun et al. *Position: Invisible Tokens, Visible Bills: The Urgent Need to Audit Hidden Operations in Opaque LLM Services*. ICML 2026. PMLR.
26. Larsen & Skou. *Bisimulation through Probabilistic Testing*. Information and Computation 94(1), 1-28, 1991. DOI.
27. Tzeng. *A Polynomial-Time Algorithm for the Equivalence of Probabilistic Automata*. SIAM Journal on Computing 21(2), 216-227, 1992. DOI.

28. Kiefer, Murawski, Ouaknine, Wachter & Worrell. *Language Equivalence for Probabilistic Automata*. CAV 2011, LNCS 6806, 526-540. DOI.

29. Kiefer, Murawski, Ouaknine, Wachter & Worrell. *On the Complexity of Equivalence and Minimisation for Q-weighted Automata*. Logical Methods in Computer Science 9(1), article 8, 2013. DOI.

30. Kiefer, Murawski, Ouaknine, Wachter & Worrell. *APEX: An Analyzer for Open Probabilistic Programs*. CAV 2012, LNCS 7358, 693-698. DOI.

Authorship and AI assistance

This upgrade used Codex and GPT-6 Luna at MAX effort for implementation, targeted research, drafting, tests, and presentation preparation. Human authors must review the implementation, claims, references, figures, authorship, and disclosure before publication. Identity/course information is carried from the repository and requires confirmation for a journal manuscript. No human sign-off or journal decision is asserted.